HDOC Day -17

Name- Aaryan Pratap Singh

Sap id - 590032842

Batch- 15

1. Algorithm

Main Program

1. Start.
2. Declare integer variable `choice`.
3. Display the menu:
   - 1 – Generate largest possible number using its digits.
   - 2 – Generate smallest possible number using its digits.
   - 3 – Swap first and last digits.
   - 4 – Check whether the number is palindrome.
4. Read `choice`.
5. Use `switch(choice)`:
   - Case 1: Call `largestNumber()`.
   - Case 2: Call `smallestNumber()`.
   - Case 3: Call `swapFirstLast()`.
   - Case 4: Call `palindrome()`.
   - Default: Display `"Invalid choice"`.
6. Stop.

Function 1: `largestNumber()`

1. Input a positive integer.
2. Extract each digit using `% 10`.
3. Store the frequency of digits `0` to `9`.
4. Arrange digits from `9` down to `0`.
5. Form the largest possible number.
6. Display the result.

Function 2: `smallestNumber()`

1. Input a positive integer.
2. Find the frequency of every digit.
3. Find the smallest **non-zero** digit and place it first.
4. Place all zeroes after the first digit.
5. Place the remaining digits in increasing order.
6. Display the smallest possible number.

This avoids a leading zero. For example, digits of `31052` form `10235`, not `01235`.

Function 3: `swapFirstLast()`

1. Input a positive integer.
2. Find the first digit.
3. Find the last digit using `% 10`.
4. Find the positional divisor of the first digit.
5. Extract the middle portion.
6. Place the last digit in the first position and the first digit in the last position.
7. Display the new number.

Function 4: `palindrome()`

1. Input a positive integer.
2. Store the original number.
3. Reverse the number digit by digit.
4. Compare the reversed number with the original number.
5. If equal, display `"Palindrome"`.
6. Otherwise, display `"Not Palindrome"`.

2. Test Case Table

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	31052	Largest possible number = 53210	Largest possible number = 53210	Pass
2	2	31052	Smallest possible number = 10235	Smallest possible number = 10235	Pass
3	3	12345	Number after swapping = 52341	Number after swapping = 52341	Pass
4	4	12321	12321 is a palindrome number	12321 is a palindrome number	Pass
5	4	12345	12345 is not a palindrome number	12345 is not a palindrome number	Pass

3. C Program

```
#include <stdio.h>

/* Function declarations */
void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void palindrome(void);
int main(void)
{
    int choice;

    printf("----- DIGIT OPERATIONS MENU -----\n");
    printf("1. Generate largest possible number\n");
```

```
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            largestNumber();
            break;
        case 2:
            smallestNumber();
            break;
        case 3:
            swapFirstLast();
            break;
        case 4:
            palindrome();
            break;
        default:
            printf("Invalid choice\n");
    }

    return 0;
}

/* Function to generate the largest possible number */
void largestNumber(void)
{
    unsigned long long n, result = 0;
    int freq[10] = {0};
    int digit, i;
    printf("Enter a positive integer: ");
    scanf("%llu", &n);
    if (n == 0)
    {
        printf("Please enter a positive integer.\n");
        return;
    }
    /* Count frequency of each digit */
    while (n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
```

```
    }
    /* Arrange digits from largest to smallest */
    for (digit = 9; digit >= 0; digit--)
    {
        for (i = 0; i < freq[digit]; i++)
        {
            result = result * 10 + digit;
        }
    }
    printf("Largest possible number = %llu\n", result);
}

/* Function to generate the smallest possible number */
void smallestNumber(void)
{
    unsigned long long n, result = 0;
    int freq[10] = {0};
    int digit, i;
    printf("Enter a positive integer: ");
    scanf("%llu", &n);
    if (n == 0)
    {
        printf("Please enter a positive integer.\n");
        return;
    }
    /* Count frequency of each digit */
    while (n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
    }
    /* Put smallest non-zero digit first */
    for (digit = 1; digit <= 9; digit++)
    {
        if (freq[digit] > 0)
        {
            result = digit;
            freq[digit]--;
            break;
        }
    }

    /* Put all zeroes after the first digit */
    for (i = 0; i < freq[0]; i++)
```

```
    {
        result = result * 10;
    }

    /* Arrange remaining digits in ascending order */
    for (digit = 1; digit <= 9; digit++)
    {
        for (i = 0; i < freq[digit]; i++)
        {
            result = result * 10 + digit;
        }
    }

    printf("Smallest possible number = %llu\n", result);
}
/* Function to swap first and last digits */
void swapFirstLast(void)
{
    unsigned long long n, temp;
    unsigned long long divisor = 1;
    unsigned long long middle, result;
    int first, last;
    printf("Enter a positive integer: ");
    scanf("%llu", &n);
    if (n == 0)
    {
        printf("Please enter a positive integer.\n");
        return;
    }
    /* Single digit number */
    if (n < 10)
    {
        printf("Number after swapping = %llu\n", n);
        return;
    }
    temp = n;
    /* Find divisor for first digit */
    while (temp >= 10)
    {
        temp = temp / 10;
        divisor = divisor * 10;
    }
    first = n / divisor;
    last = n % 10;
```

```
    /* Extract middle digits */
    middle = (n % divisor) / 10;
    /* Form number after swapping */
    result = (unsigned long long)last * divisor
             + middle * 10
             + first;
    printf("Number after swapping = %llu\n", result);
}
/* Function to check palindrome */
void palindrome(void)
{
    unsigned long long n, temp, reverse = 0;
    printf("Enter a positive integer: ");
    scanf("%llu", &n);
    if (n == 0)
    {
        printf("Please enter a positive integer.\n");
        return;
    }
    temp = n;
    /* Reverse the number */
    while (temp > 0)
    {
        reverse = reverse * 10 + temp % 10;
        temp = temp / 10;
    }
    /* Compare original and reversed number */
    if (reverse == n)
    {
        printf("%llu is a palindrome number.\n", n);
    }
    else
    {
        printf("%llu is not a palindrome number.\n", n);
    }
}
```

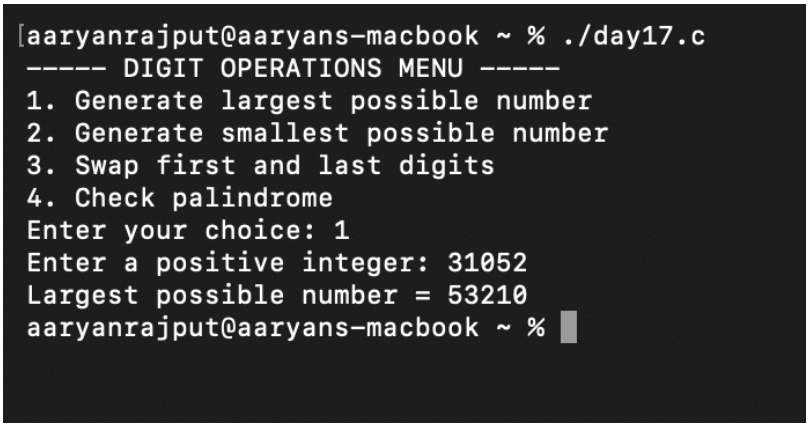
4. Compilation

The program was compiled

```
aaryanrajput@aaryans-macbook ~ % nano day17.c
aaryanrajput@aaryans-macbook ~ % clang day17.c -o day17.c
aaryanrajput@aaryans-macbook ~ %
```

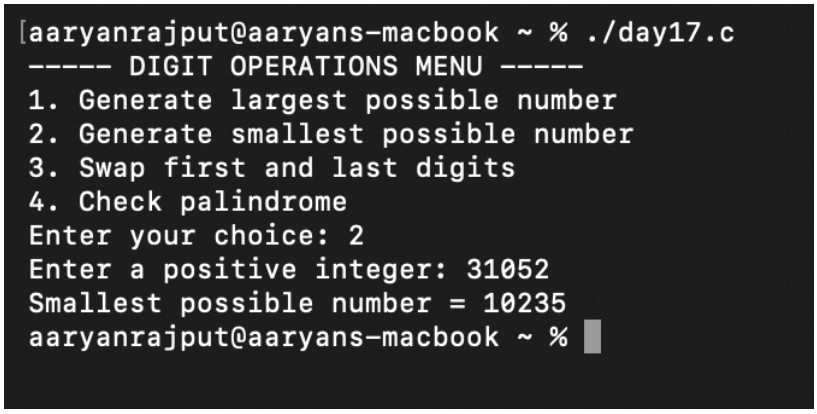
5. Execution and Testing

Test 1 — Largest Number



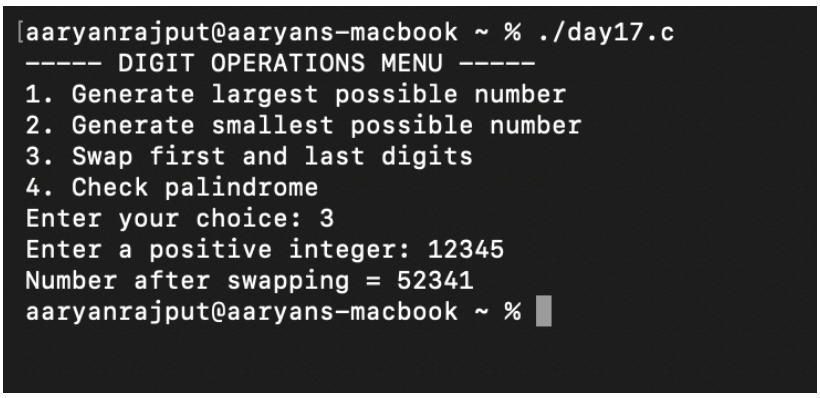
Result: PASS

Test 2 — Smallest Number



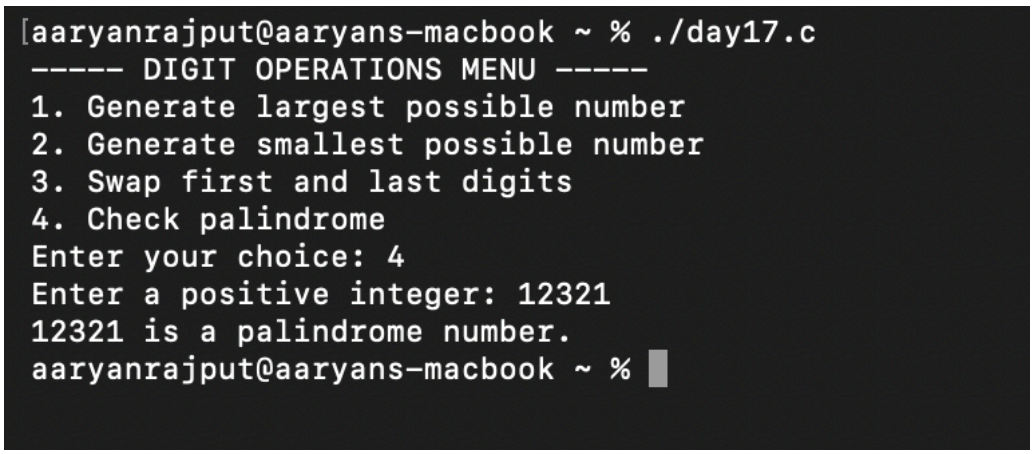
Result: PASS

Test 3 — Swap First and Last Digits



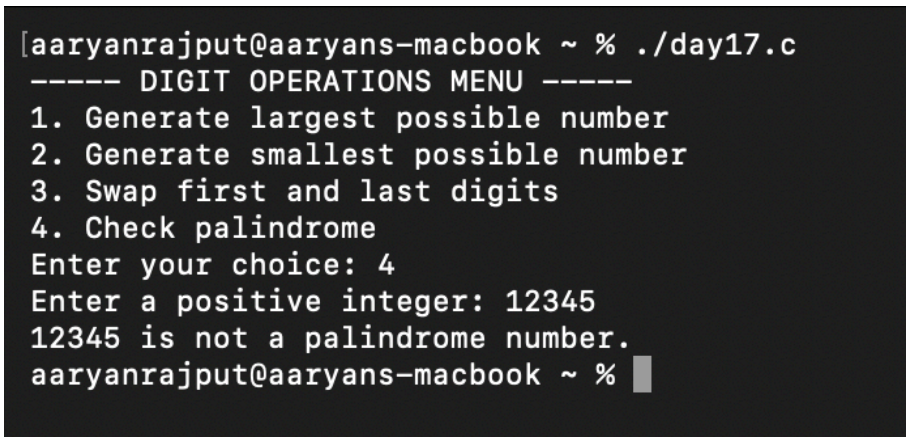
Result: PASS

Test 4 — Palindrome



Result: PASS

Test 5 — Not Palindrome



Result: PASS

All **5 prepared test cases passed successfully**. The program satisfies the requirement of using `switch-case`, separate `void`functions, and simple function calls such as `largestNumber();` without passing arguments.